

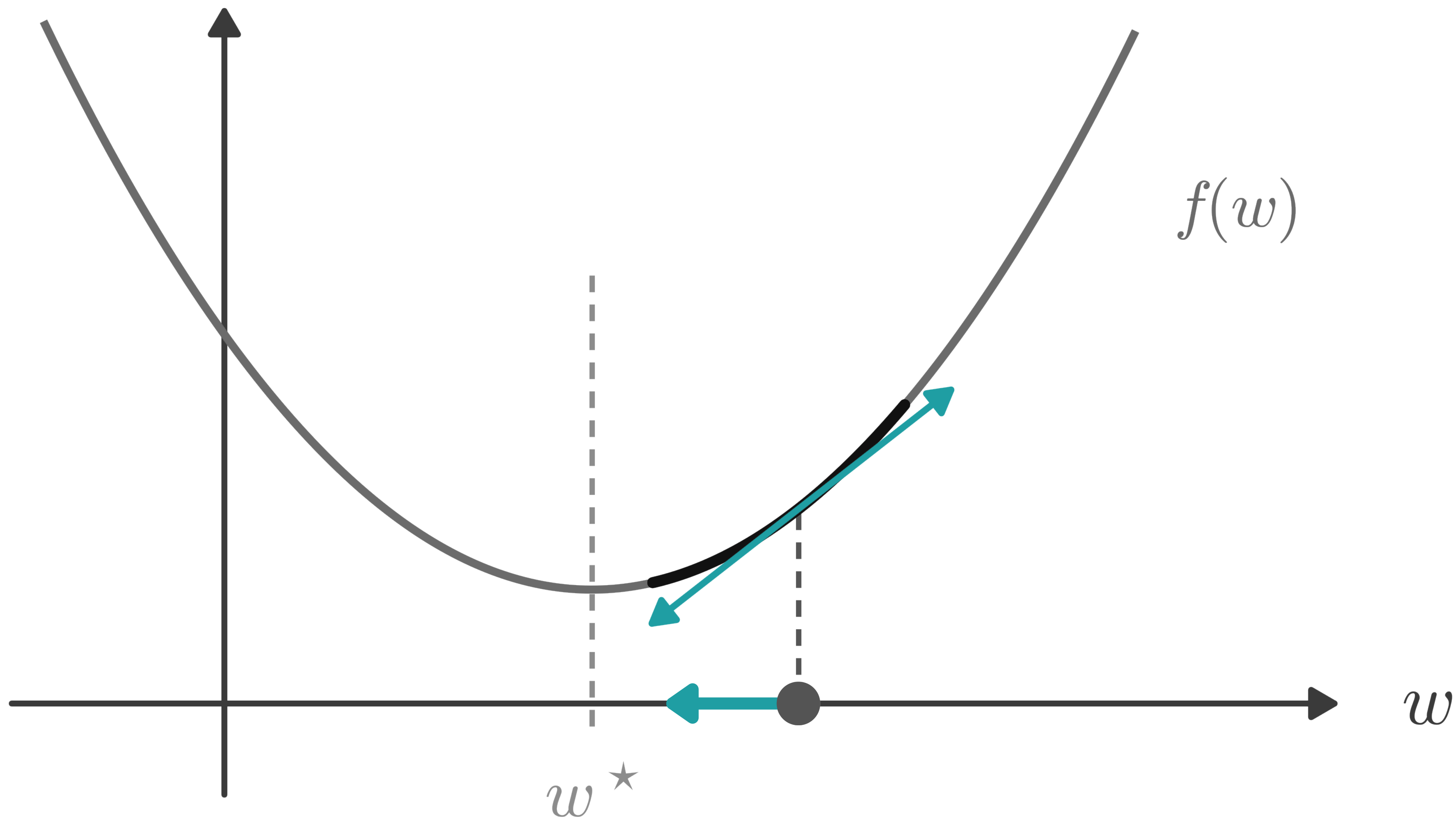
Cornell CS 3/5780

Introduction to Machine Learning

Gradient Descent

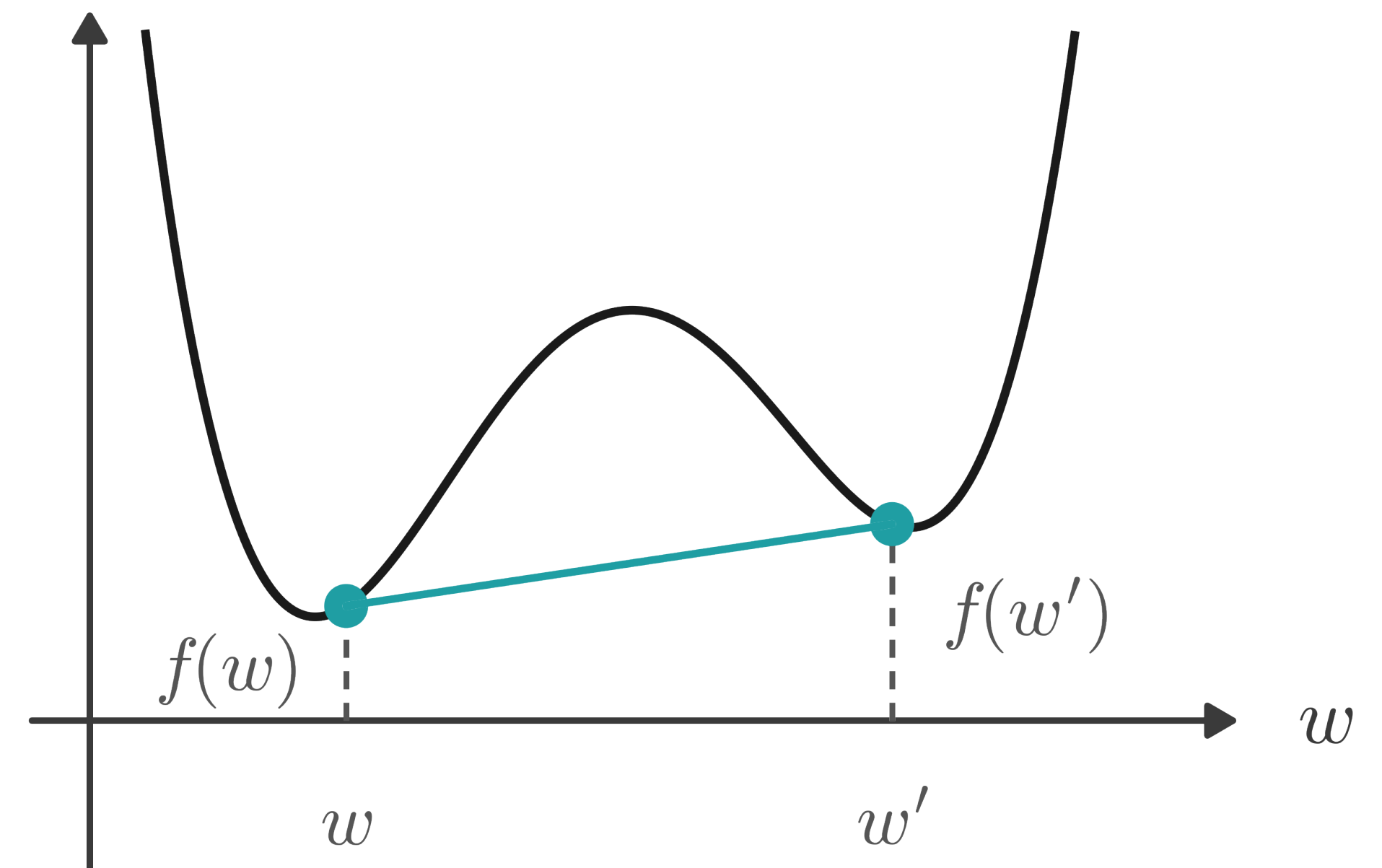
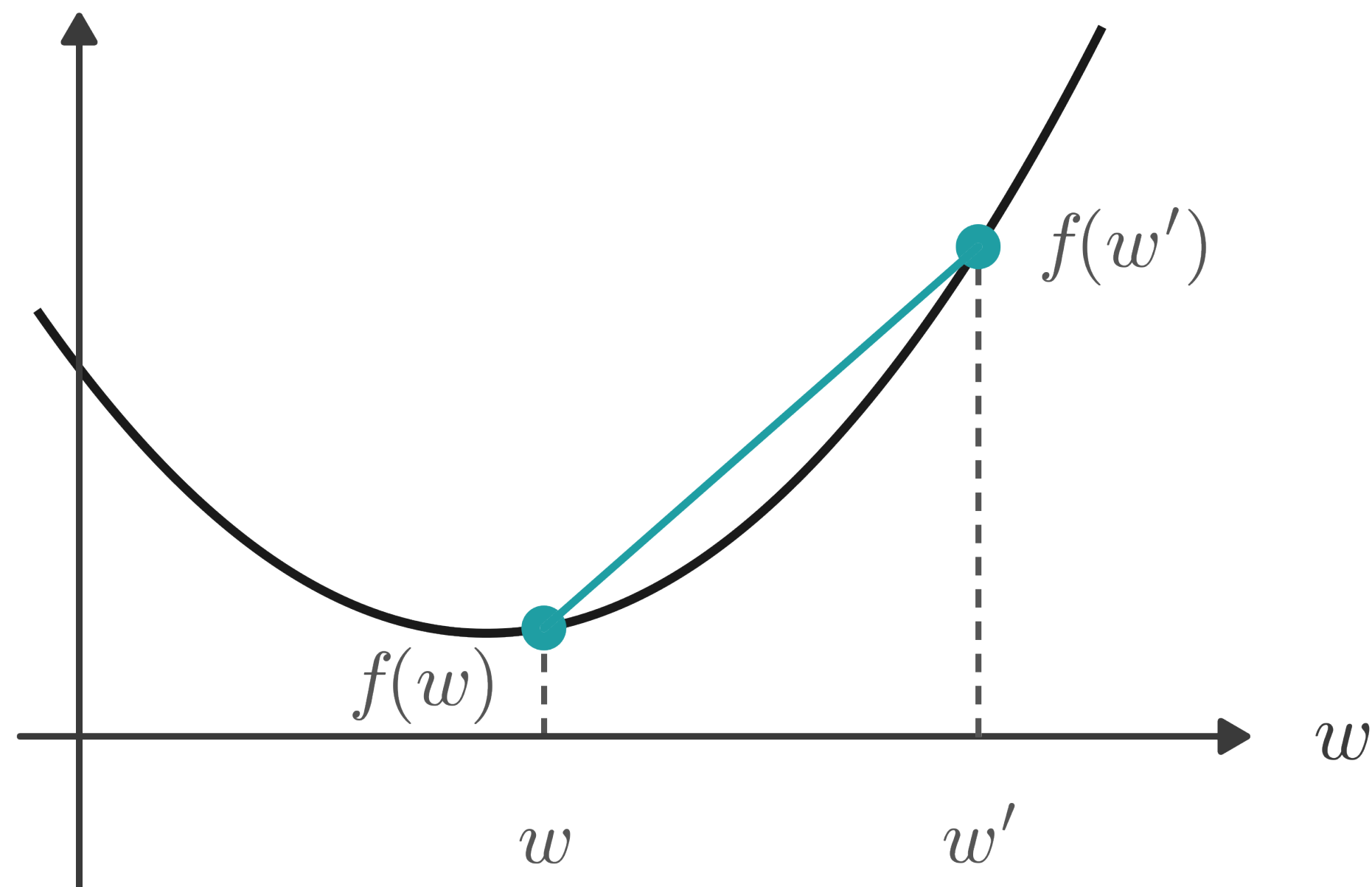
Instructors: Karthik Sridharan and John Thickstun

Warm up: 1D minimization



Minima & Convexity

- Definition: A multivariate function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is **convex** if
$$f(\alpha \vec{w} + (1 - \alpha) \vec{w}') \leq \alpha f(\vec{w}) + (1 - \alpha) f(\vec{w}').$$
- For a convex function, all local minima are global minima.



Partial Derivatives

- The **(partial) derivatives** of a multivariate function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ at a point $\vec{w} \in \mathbb{R}^d$ are the single-variable derivatives of f for each coordinate w_j , holding the other coordinates fixed:

$$\frac{\partial f(\vec{w})}{\partial w_j} = \frac{d}{dw_j} f(w_1, \dots, w_{j-1}, w_j, w_{j+1}, \dots, w_d).$$

$$\begin{aligned} \blacktriangleright \text{Example: } f(\vec{w}) = \vec{w} \cdot \vec{w} = \sum_{j=1}^d w_j^2 &\implies \frac{\partial f(\vec{w})}{\partial w_j} = 2w_j \\ &\implies \nabla f(\vec{w}) = 2\vec{w} \end{aligned}$$

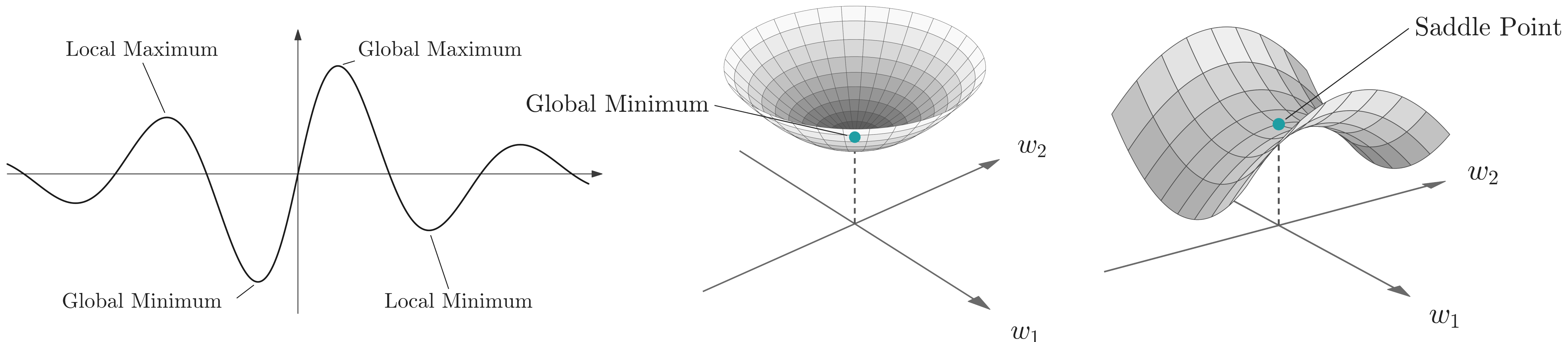
$$\begin{aligned} \blacktriangleright \text{Example: } f(\vec{w}) = \vec{x} \cdot \vec{w} = \sum_{j=1}^d x_j w_j &\implies \frac{\partial f(\vec{w})}{\partial w_j} = x_j \\ &\implies \nabla f(\vec{w}) = \vec{x} \end{aligned}$$

Gradients

- Definition: the **gradient** of a smooth multivariate function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ at a point $\vec{w}$ is the vector of (partial) derivatives along each input coordinate of f :

$$\nabla f(\vec{w}) = \left(\frac{\partial f(\vec{w})}{\partial w_1}, \frac{\partial f(\vec{w})}{\partial w_2}, \dots, \frac{\partial f(\vec{w})}{\partial w_d} \right).$$

- Definition: **critical points** of $f : \mathbb{R}^d \rightarrow \mathbb{R}$ are $\vec{w}^*$ such that $\nabla f(\vec{w}^*) = 0$.



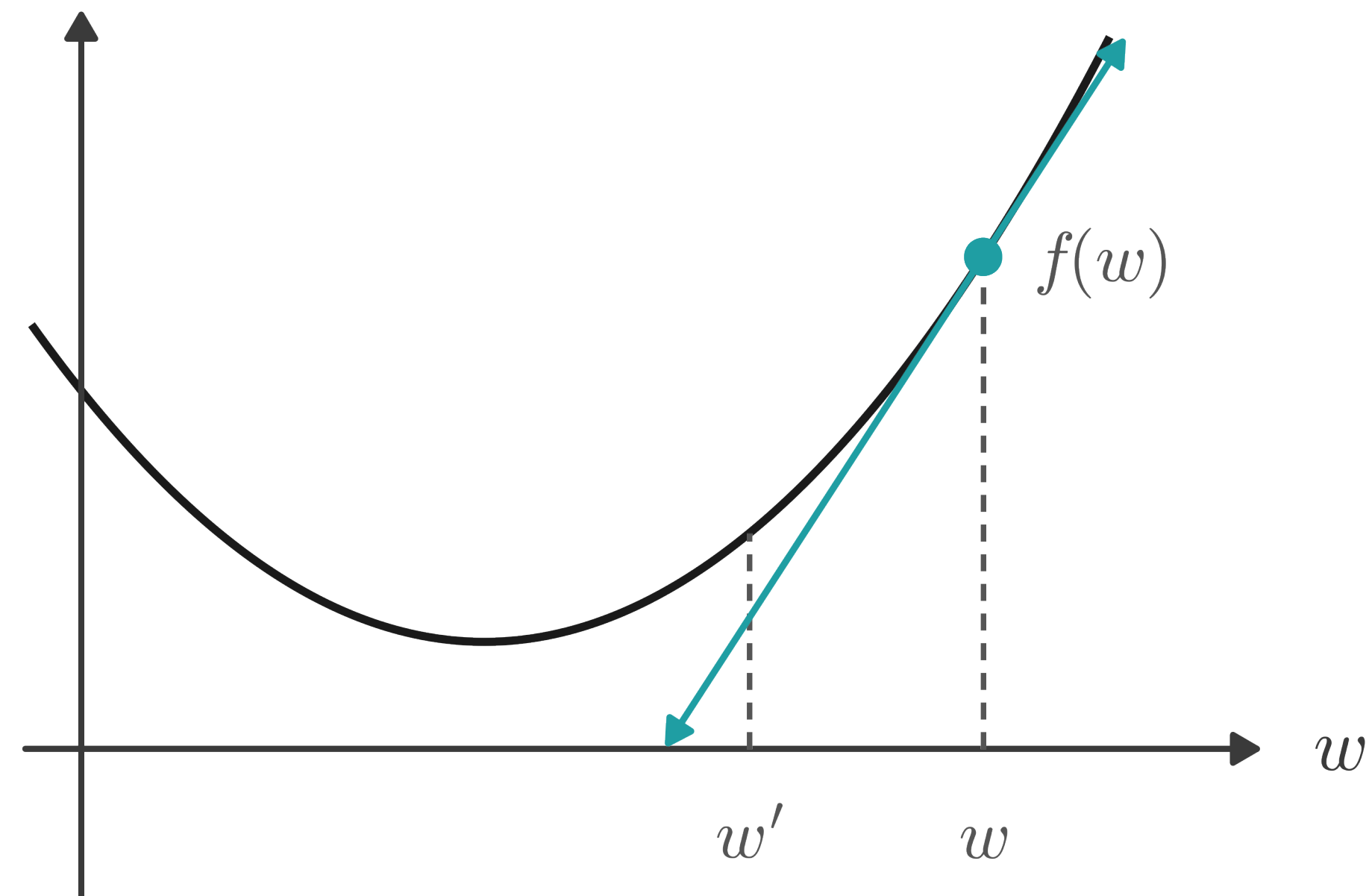
Smooth Convex Functions

- A smooth multivariate function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is **convex** if and only if

$$f(\vec{w}') \geq f(\vec{w}) + \nabla f(\vec{w}) \cdot (\vec{w}' - \vec{w}).$$

- For a convex function, all critical points are global minima.

➡ $f(\vec{w})$ is minimized when $\nabla f(\vec{w}) = 0$.



Gradient Descent

Finding the minimum of a smooth convex function

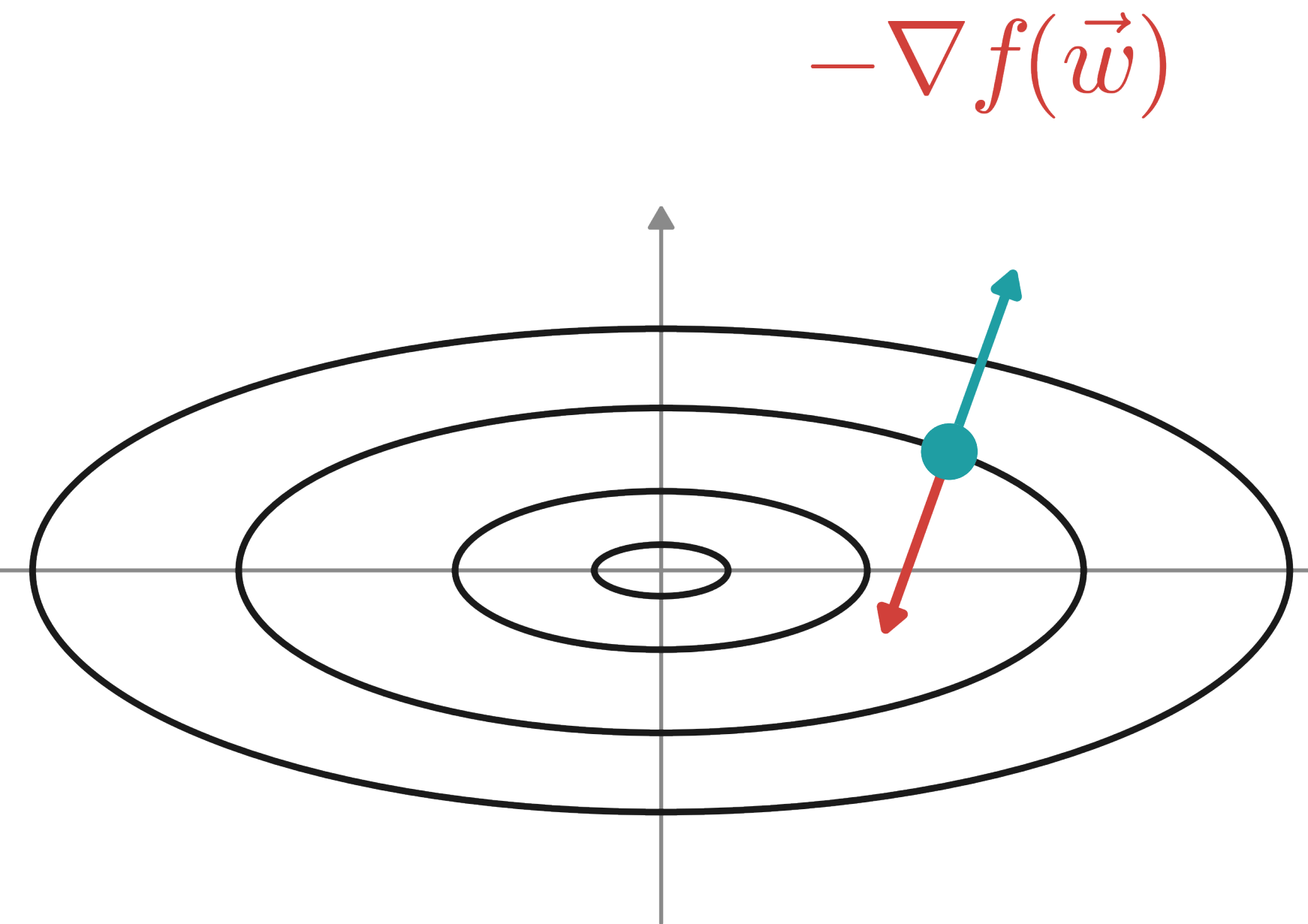
Algorithm: Gradient Descent

Input: a function f , time T , step size η_t

Initialize: $\vec{w}^{(0)} = (0, \dots, 0)$

Loop: for $t = 1, 2, \dots, T$:

$$\vec{w}^{(t+1)} = \vec{w}^{(t)} - \eta_t \nabla f(\vec{w}^{(t)})$$



- Theoretical guarantees for convex f (and some special non-convex)
- Step size η_t is important in theory and practice

Learning Linear Models

- For training dataset $\mathcal{S} = \{(\vec{x}_1, y_1), \dots, (\vec{x}_n, y_n)\}$ many learning problems can be written as

$$\min_{\vec{w}} \mathcal{L}_{\mathcal{S}}(\vec{w}) = \frac{1}{n} \sum_{i=1}^n \ell(\vec{w} \cdot \vec{x}_i, y_i) + R(\vec{w}).$$

- For example, linear regression (ordinary least squares):

$$\mathcal{L}_{\mathcal{S}}(\vec{w}) = \frac{1}{n} \sum_{i=1}^n (\vec{w} \cdot \vec{x}_i - y_i)^2$$

$$\implies \nabla \mathcal{L}_{\mathcal{S}}(\vec{w}) = \nabla \frac{1}{n} \sum_{i=1}^n (\vec{w} \cdot \vec{x}_i - y_i)^2 = \frac{1}{n} \sum_{i=1}^n \nabla (\vec{w} \cdot \vec{x}_i - y_i)^2 = \frac{1}{n} \sum_{i=1}^n 2 (\vec{w} \cdot \vec{x}_i - y_i) \vec{x}_i$$

Gradient Descent for Linear Regression

Algorithm: Gradient Descent

Input: a function f , time T , step size η_t

Initialize: $\vec{w}^{(0)} = (0, \dots, 0)$

Loop: for $t = 1, 2, \dots, T$:

$$\vec{w}^{(t+1)} = \vec{w}^{(t)} - \frac{2\eta_t}{n} \sum_{i=1}^n \left(\vec{w}^{(t)} \cdot \vec{x}_i - y_i \right) \vec{x}_i$$

- Recall: the minimizer of linear regression is $\vec{w}^* = (XX^T)^{-1}XY$.
- Why not just compute $\vec{w}^*$ using this closed form solution?

Stopping Criterion

- How do we set T ?

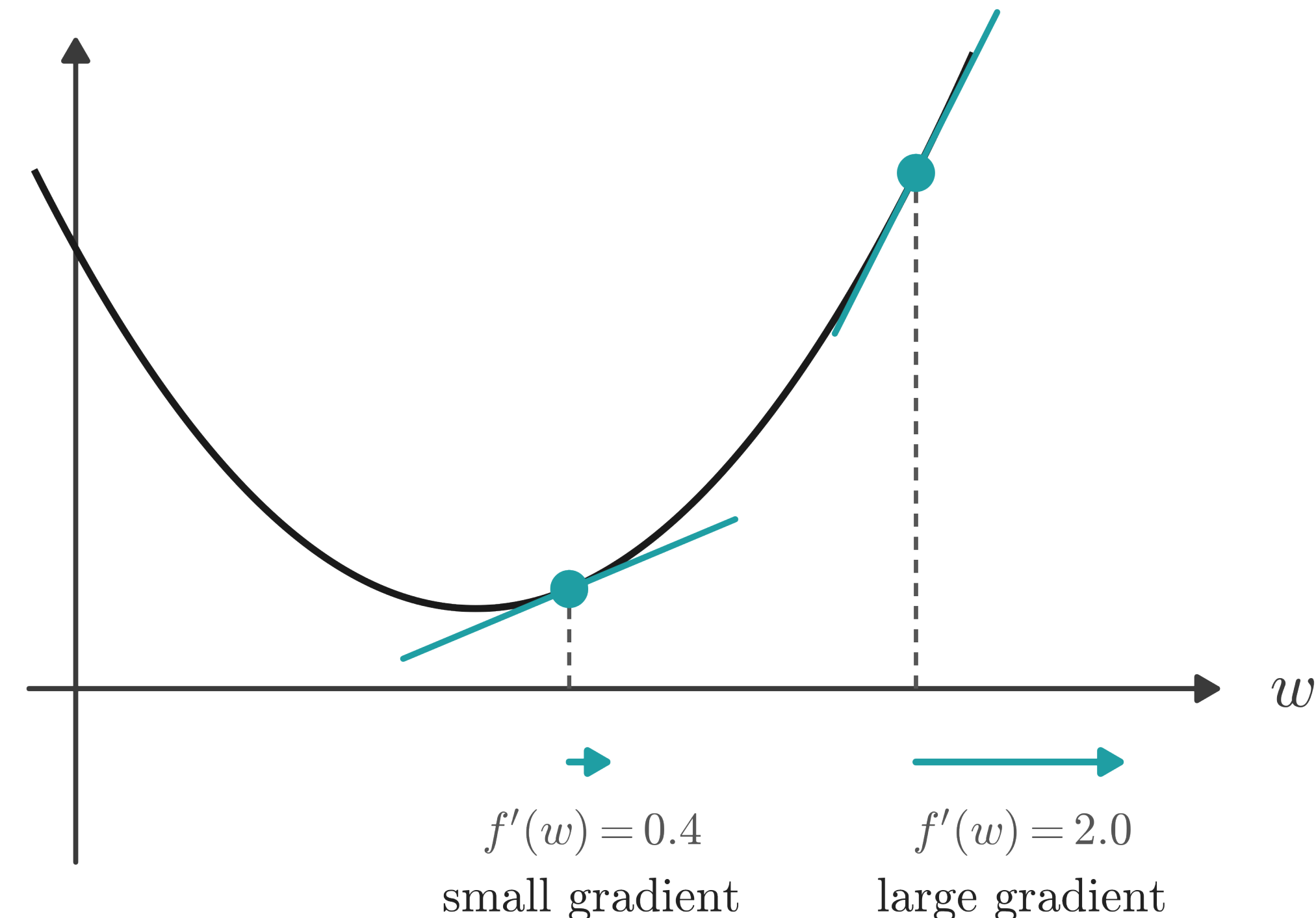
Algorithm: Gradient Descent

Input: a function f , tolerance δ , step size η_t

Initialize: $\vec{w}^{(0)} = (0, \dots, 0)$

Loop: while $\|\nabla f(\vec{w}^{(t)})\| \geq \delta$

$$\vec{w}^{(t+1)} = \vec{w}^{(t)} - \eta_t \nabla f(\vec{w}^{(t)})$$



- Look at $\|\nabla f(\vec{w}^{(t)})\|$ and stop once the gradients get small.

Large-Scale Optimization

- Every step of gradient descent computes $\nabla \ell(\vec{w} \cdot \vec{x}_i, y_i)$ for every pair of data $(\vec{x}_i, y_i) \in \mathcal{S}$:

$$\vec{w}^{(t+1)} = \vec{w}^{(t)} - \frac{\eta_t}{n} \sum_{i=1}^n \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}_i, y_i) + \nabla R(\vec{w}^{(t)}).$$

- Not a great idea when the dataset is large (large n).
- Idea: subsample the dataset to get an estimate of the gradient:

$$\frac{1}{k} \sum_{i \in \mathcal{S}'} \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}_i, y_i) \approx \frac{1}{n} \sum_{i=1}^n \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}_i, y_i)$$

where $\mathcal{S}'$ is a subset of k elements of $\mathcal{S}$ chosen at random.

Stochastic Gradient Descent

Algorithm: SGD for Regularized Linear Models

Input: a function f , time T , step size η_t

Initialize: $\vec{w}^{(0)} = (0, \dots, 0)$

Loop: for $t = 1, 2, \dots, T$:

Select: $\vec{x}, y \in \mathcal{S}$ uniformly at random

$$\vec{w}^{(t+1)} = \vec{w}^{(t)} - \eta_t \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}, y) + \eta_t R(\vec{w}^{(t)})$$

- In practice: shuffle the dataset and choose $(\vec{x}, y)$ without replacement
- Reshuffle and start over after exhausting the dataset (an epoch)

Justification for SGD

Select: $\vec{x}, y \in \mathcal{S}$ uniformly at random

$$\vec{w}^{(t+1)} = \vec{w}^{(t)} - \eta_t \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}, y) + \eta_t R(\vec{w}^{(t)})$$

Justification for SGD

Select: $\vec{x}, y \in \mathcal{S}$ uniformly at random

$$\vec{w}^{(t+1)} = \vec{w}^{(t)} - \eta_t \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}, y) + \eta_t R(\vec{w}^{(t)})$$

$$\nabla \mathcal{L}_{\mathcal{S}}(\vec{w}^{(t)}) = \frac{1}{n} \sum_{i=1}^n \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}_i, y_i) + \nabla R(\vec{w}^{(t)})$$

- Why does this work?

$$\vec{g}^{(t)} = \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}, y) + \nabla R(\vec{w}^{(t)})$$

$$\mathbb{E}[\vec{g}^{(t)} | \vec{w}^{(t)}] = \mathbb{E}[\nabla \ell(\vec{w}^{(t)} \cdot \vec{x}, y) | \vec{w}^{(t)}] + \nabla R(\vec{w}^{(t)})$$

$$= \sum_{i=1}^n P(\vec{x}_i, y_i) \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}_i, y_i) + \nabla R(\vec{w}^{(t)})$$

$$= \sum_{i=1}^n \frac{1}{n} \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}_i, y_i) + \nabla R(\vec{w}^{(t)}) = \nabla \mathcal{L}_{\mathcal{S}}(\vec{w}^{(t)})$$

SGD for SVM

- Recall the SVM objective:

$$\min_{\vec{w}} \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i \vec{w} \cdot \vec{x}_i) + \frac{\lambda}{2} \vec{w} \cdot \vec{w}$$

- Regularizer: $\nabla \frac{\lambda}{2} \vec{w} \cdot \vec{w} = \lambda \vec{w}$

- Hinge loss: $\nabla [\max(0, 1 - y_i \vec{w} \cdot \vec{x}_i)] = \nabla \begin{cases} 0 & y_i \vec{w} \cdot \vec{x}_i > 1 \\ 1 - y_i \vec{w} \cdot \vec{x}_i & y_i \vec{w} \cdot \vec{x}_i \leq 1 \end{cases}$
 $= \begin{cases} 0 & y_i \vec{w} \cdot \vec{x}_i > 1 \\ -y_i \vec{x}_i & y_i \vec{w} \cdot \vec{x}_i \leq 1 \end{cases}$
 $= -y_i \vec{x}_i \mathbf{1}_{\{y_i \vec{w} \cdot \vec{x}_i \leq 1\}}$

SGD for SVM

- SVM gradient: $\frac{1}{n} \sum_{i=1}^n$

- Gradient descent update:

$$\vec{w} \leftarrow \vec{w} + \frac{\eta}{n} \sum_{i=1}^n y_i \vec{x}_i \mathbf{1}_{\{y_i \vec{w} \cdot \vec{x}_i \leq 1\}} - \eta \lambda \vec{w}$$

- SVM SGD update: select $(\vec{x}, y) \in \mathcal{S}$ uniformly at random and

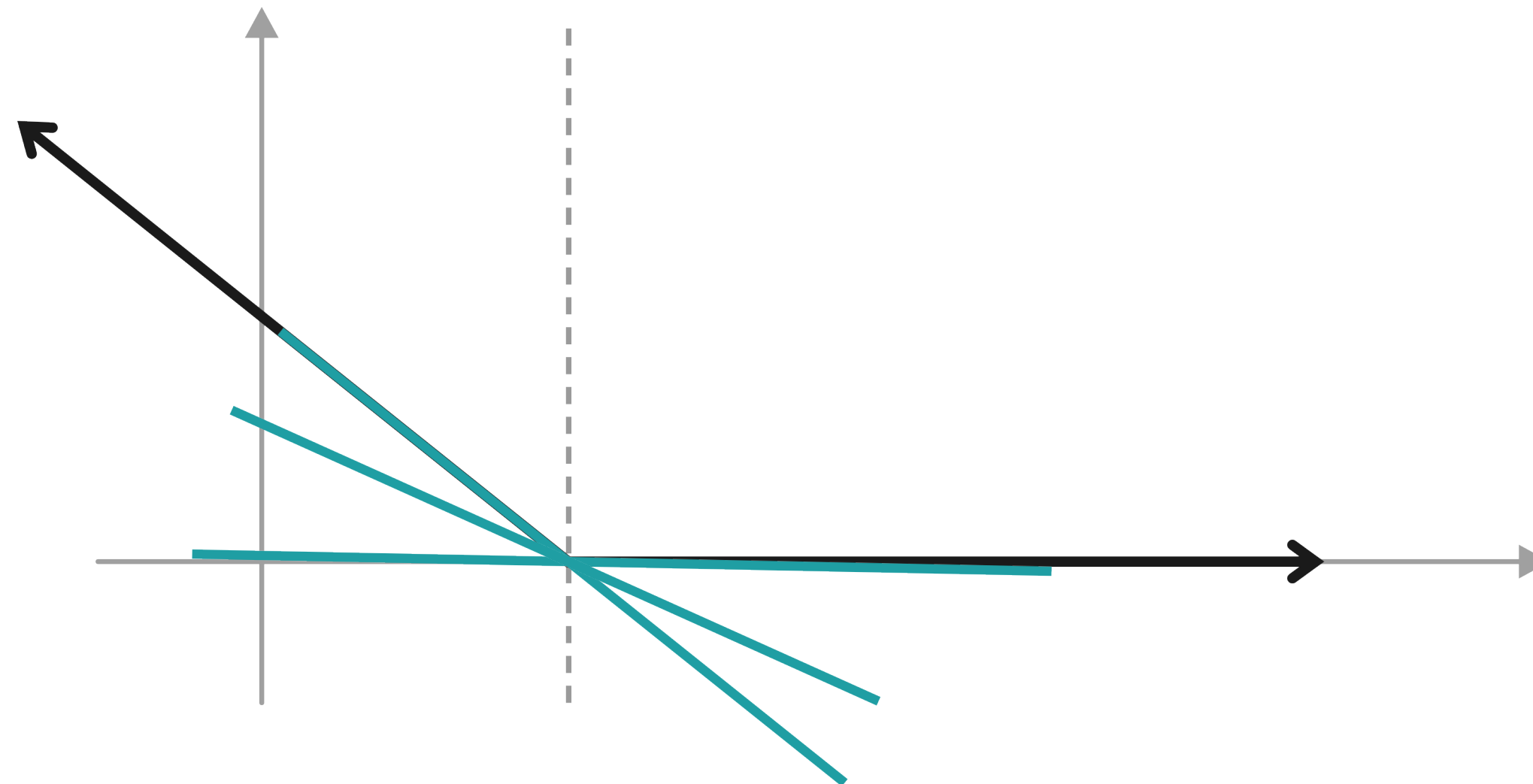
$$\text{if } y \vec{w} \cdot \vec{x} \leq 1 \text{ then } \vec{w} \leftarrow (1 - \eta \lambda) \vec{w} + \eta y \vec{x} \text{ else } \vec{w} \leftarrow (1 - \eta \lambda) \vec{w}$$

- Compare to Perceptron (SVM has stricter margin and “shrinkage”):

$$\text{if } y \vec{w} \cdot \vec{x} \leq 0 \text{ then } \vec{w} \leftarrow \vec{w} + y \vec{x} \quad -y_i \vec{x}_i \mathbf{1}_{\{y_i \vec{w} \cdot \vec{x}_i \leq 1\}}$$

Non-smooth functions?

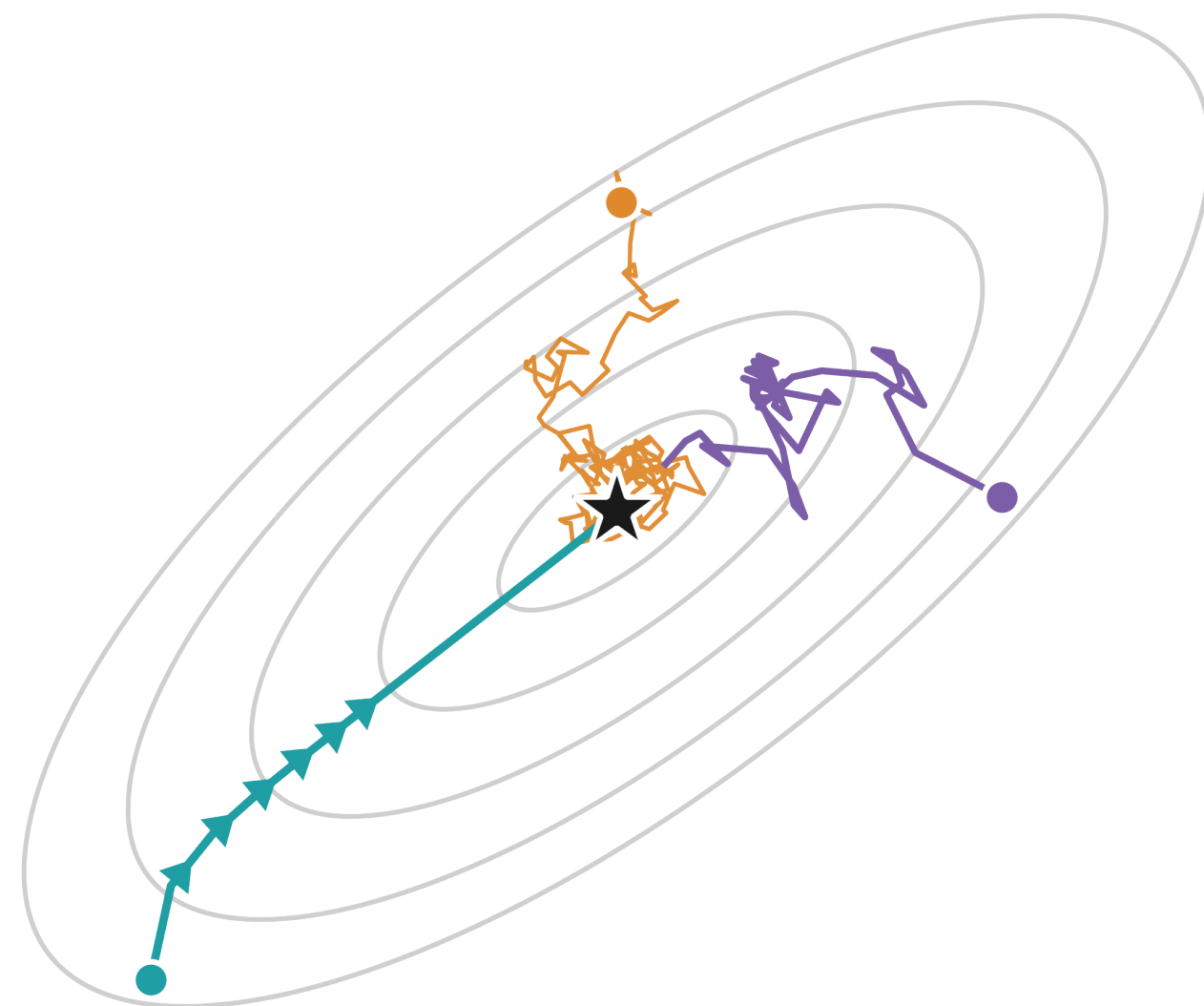
- What to do with functions f that don't have a gradient everywhere?
- A **sub-gradient** is any tangent that lies below f
- Every convex function has at least one subgradient at all points (even if they are non-smooth).
- We can run gradient descent / SGD with sub-gradients!



Mini-Batching

- In theory: SGD with a single data point per step is sufficient
- In practice: better to estimate the gradient with more than one point
- A **mini-batch** is the subset of training data used to estimate a gradient
- Select a random subset $\mathcal{S}_k \subset \mathcal{S}$ (mini-batch of size k) and compute

$$\vec{w}^{(t+1)} = \vec{w}^{(t)} - \frac{\eta_t}{k} \sum_{i \in \mathcal{S}_k} \nabla \ell(\vec{w}^{(t)} \cdot \vec{x}_i, y_i) + \nabla R(\vec{w}^{(t)})$$



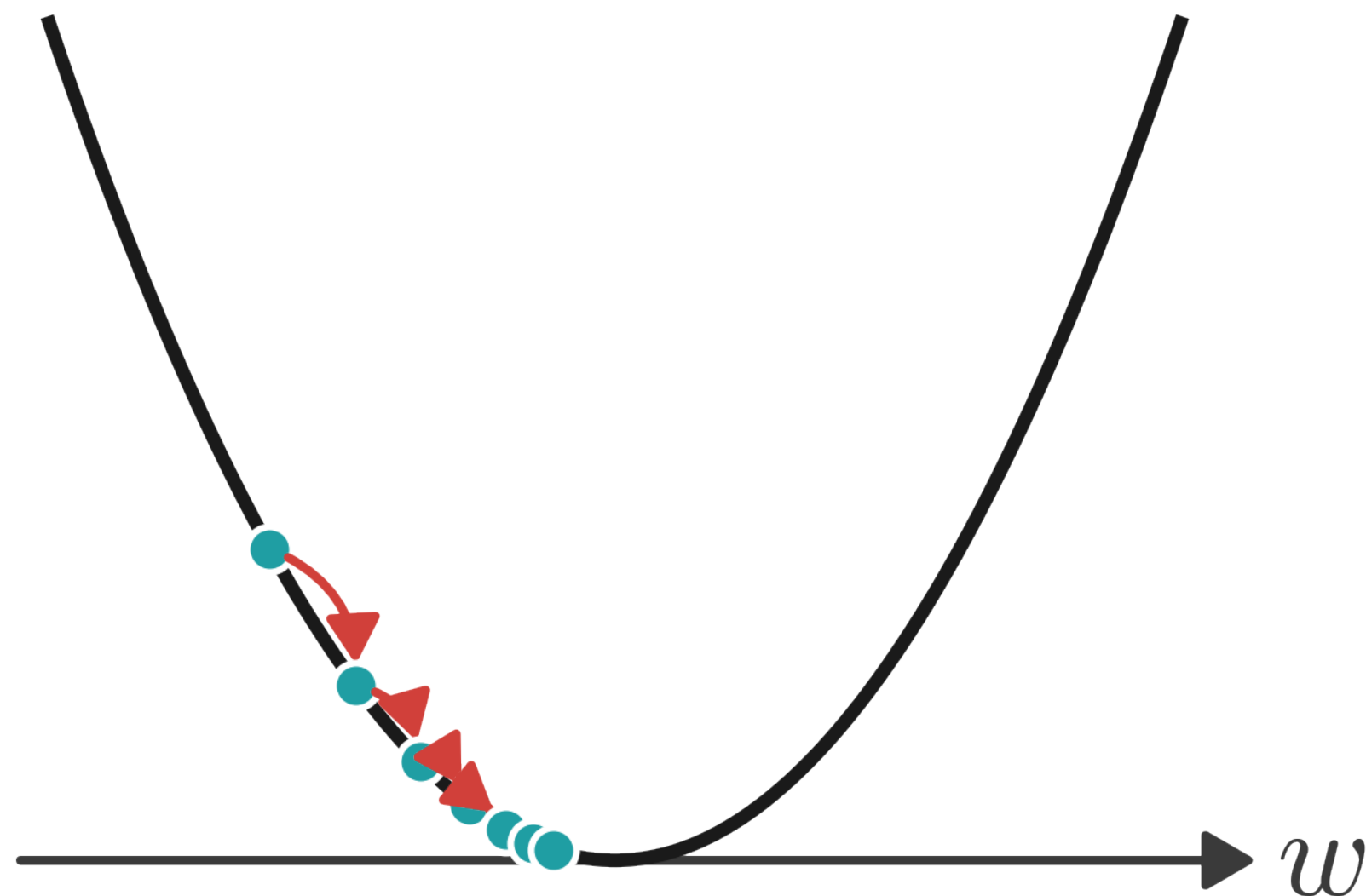
- Batch gradient descent
- Mini-batch gradient descent
- Stochastic gradient descent
- ★ Minimum

Step Size (Learning Rate)

- Too small step size? Slow progress towards the minimum.
- Too large step size? Overshooting the minimum.

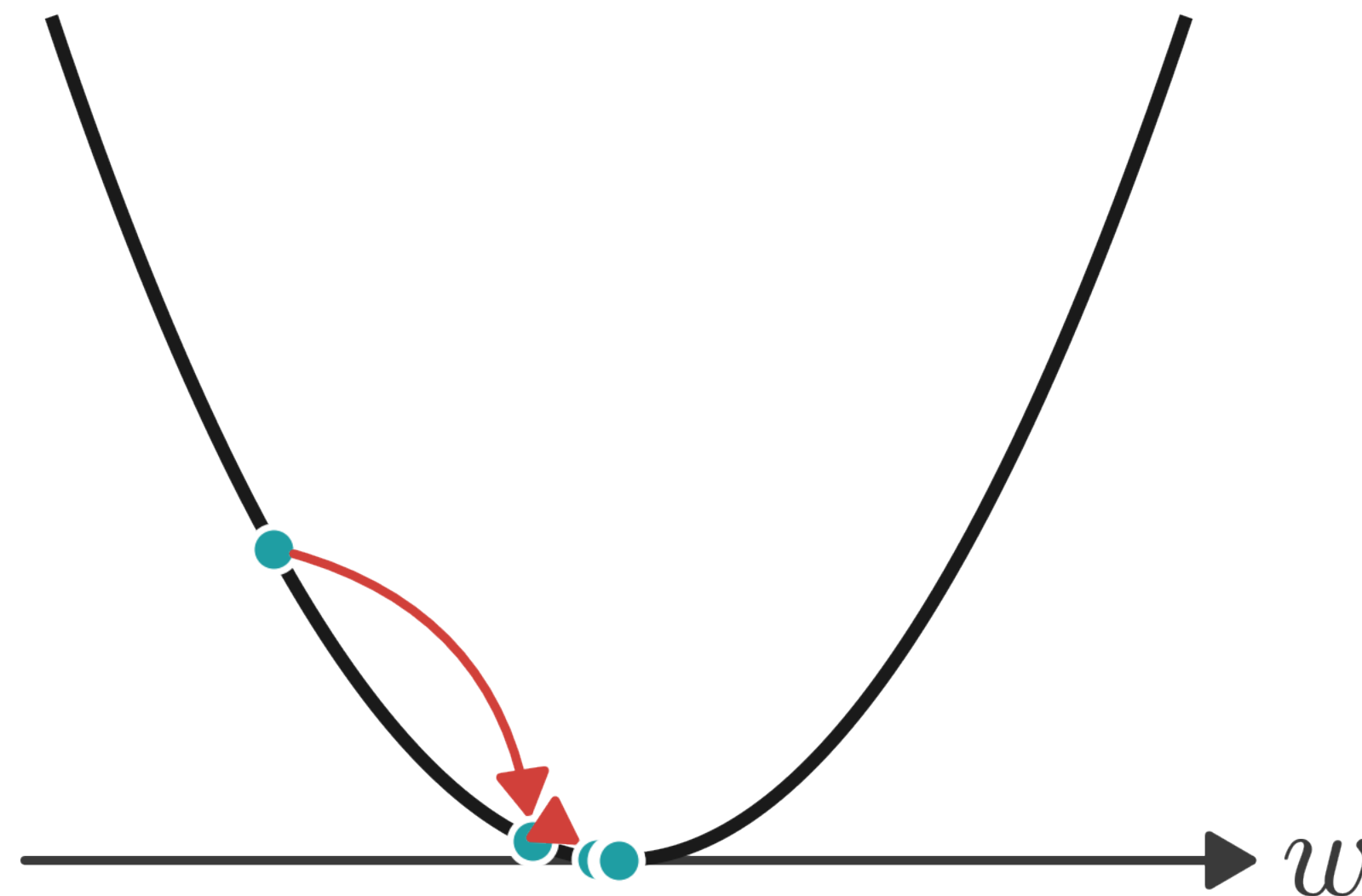
step size too small

$$\eta = 0.25$$



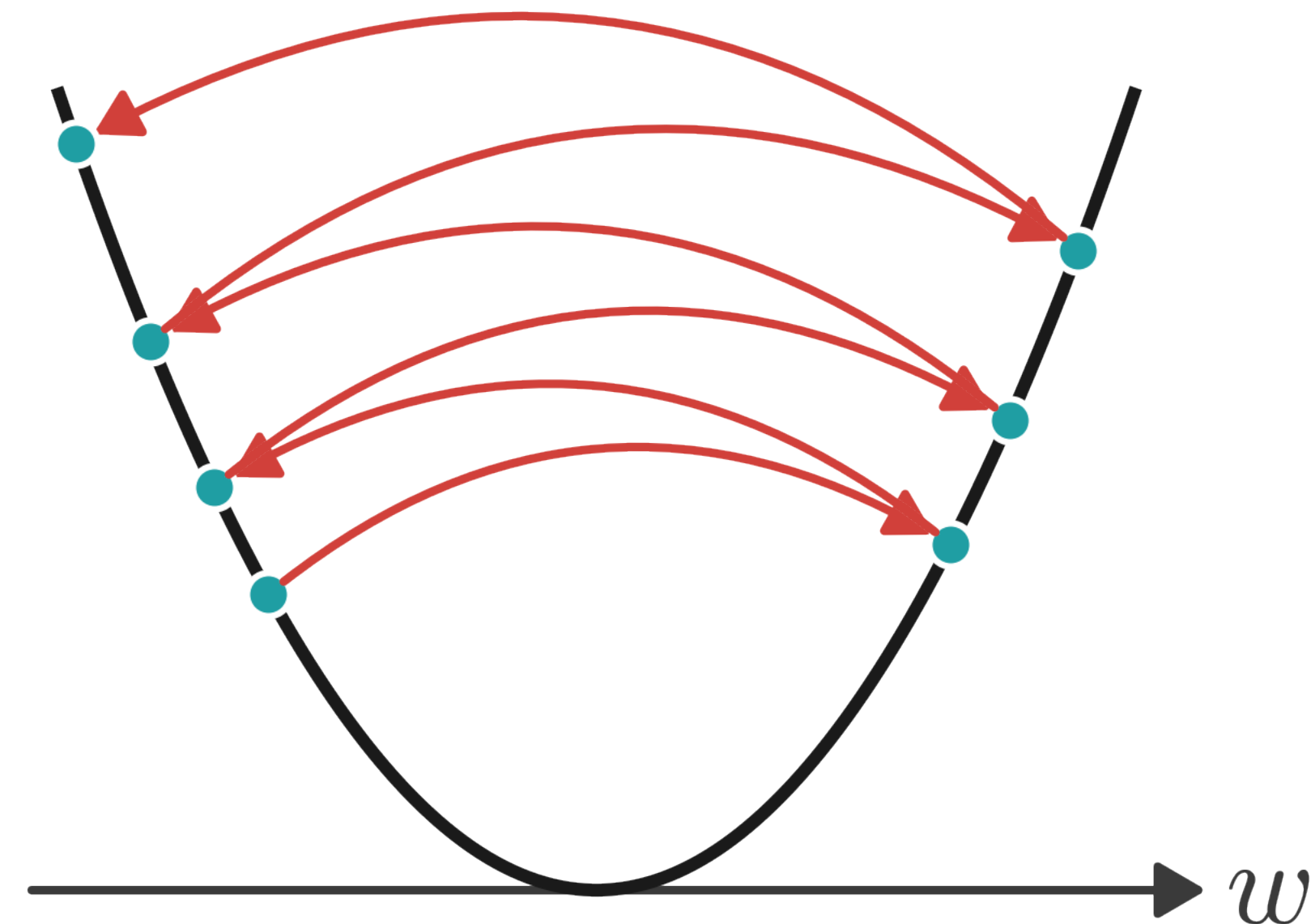
step size just right

$$\eta = 0.75$$



step size too large

$$\eta = 2.08$$



Adaptive Gradients

- **AdaGrad:** Change the step size for each coordinate based on previous gradients. For all coordinates $i = 1, \dots, d$:

➡ Derivative is $g_{i,t} = \frac{\partial \mathcal{L}(\vec{w}^{(t)})}{\partial w_i} = \left[\nabla \mathcal{L}(\vec{w}^{(t)}) \right]_i$

➡ AdaGrad update is $w_i^{(t+1)} = w_i^{(t)} - \eta \frac{g_{i,t}}{\sqrt{\varepsilon + \sum_{k=1}^t g_{i,k}^2}}$

- Scale gradient for i 'th coordinate by sum of squares of past gradients.
- Steep axes will get small learning rates; shallow axes get large ones.
- Epsilon avoids pathological division by zero.